CC1002 Introducción a la Programación - Sección 1
Tarea 3

Prof. Benjamin Bustos

Fecha de entrega: lunes 16 de noviembre de 2026 a las 23:59

Es obligatorio el uso de la receta de diseño visto en cátedra para implementar todas las funciones.

La tarea puede realizarse en forma individual o en un grupo de hasta dos personas. No olvide leer el documento de Política de Colaboración publicado en U-Cursos.

Manejo de inventario de materiales para un videojuego

Una importante empresa desarrolladora de videojuegos le solicita a Ud. programar parte de su más reciente éxito comercial, el juego "Castle Day". Este es un juego multi-plataforma y multi-jugador, en donde las personas compiten por recursos y se enfrentan en batallas épicas. Parte del objetivo del juego es hacer el mejor uso de los recursos del ambiente, para construir estructuras que permitan obtener ventaja y así aumentar las probabilidades de ganar el juego.

Denominaremos *Player* a una persona jugadora de Castle Day. Una parte importante del juego consiste en recopilar recursos, que luego pueden utilizarse para construir distintos tipos de estructuras. Los materiales que se pueden recopilar son: madera (que se obtiene de árboles en el mapa), piedra (que se obtiene de rocas en el mapa), y metal (que se obtiene de restos metálicos como autos, maquinarias, etc., en el mapa). Player mantiene un inventario con la lista de los materiales que ha ido recopilando, guardando el tipo de material y la cantidad (tablas en el caso de madera, rocas en el caso de piedra, y lingotes en el caso de metal). Por ejemplo, Player pudo haber recopilado primero tres tablas de madera, luego dos rocas, luego dos

tablas de madera, luego dos lingotes de metal, y finalmente una roca. Para este ejemplo, su inventario queda almacenado en una lista como sigue:

```
[('madera', 3), ('piedra', 2), ('madera', 2), ('metal', 2), ('piedra', 1)]
```

Luego, con estos materiales, Player puede construir distintos tipos de estructuras: murallas, pisos, escaleras, y pirámides. Cada una de estas estructuras cumple alguna función relevante en el juego y permiten definir distintos tipos de estrategia para ganar el juego. La cantidad de material necesario para construir una estructura depende de lo que se esté construyendo y del material utilizado. Estructuras construidas con madera son menos costosas de construir, pero poco duraderas. Las estructuras de piedra son más resistentes, pero requieren más material para poder construirlas. El metal es el mejor material de todos, pero es más escaso y requiere de más tiempo para construir la estructura respectiva. La lista de materiales necesarios para construir cada tipo de estructura es la siguiente:

- Muralla: 5 tablas de madera, o 7 rocas, o 4 lingotes de metal.
- Piso: 6 tablas de madera, o 9 rocas, o 5 lingotes de metal.
- Escalera: 3 tablas de madera, o 4 rocas, o 3 lingotes de metal.
- Pirámide: 10 tablas de madera, o 14 rocas, o 7 lingotes de metal.

Lo que requiere la importante empresa desarrolladora de videojuegos es que Ud. implemente el manejo del inventario de Player, agregando materiales al inventario a medida que Player los va recopilando, y sacando material del inventario cuando Player construye alguna estructura. Note que Player puede intentar construir una estructura sin tener el material suficiente, en cuyo caso NO debe construirse ni tampoco se debe quitar material del inventario en dicho caso.

Parte 1

Implemente esta parte en el módulo `Tarea3Parte1.py`.

Se usará la siguiente estructura mutable para representar los materiales que recopila Player:

```
# material: tipo(str) cantidad(int)
# tipo puede ser 'madera', 'piedra', 'metal'
estructura.mutable('material', 'tipo cantidad')
```

El inventario corresponde a una variable de estado implementado con una lista de Python de datos de tipo `material`, inicialmente vacía.

Implemente un módulo de Python que provea los siguientes procedimientos:

1. `agregarInventario(tipo, cantidad)`: agrega al final del inventario el material y cantidad respectivos. Si se intenta agregar un tipo de material desconocido, o la cantidad es un valor $<= 0$, se debe ignorar y no se agrega al inventario.

2. `construirEstructura(tipoEstructura, tipoMaterial)`: construye la estructura definida por `tipoEstructura` ('muralla', 'piso', 'escalera', o 'piramide') y el tipo de material ('madera', 'piedra', 'metal'), descontando el material utilizado del inventario partiendo desde el principio de la lista. Note que la cantidad de material necesario para construir la estructura puede estar en varias posiciones del inventario. Si luego de construida la estructura, una posición del inventario queda con una cantidad de material igual a cero, se debe eliminar del inventario. En caso que el tipo de estructura no exista, el tipo de material es desconocido, o no hay suficiente material en el inventario para construir la estructura, no se hace nada y no se quita material del inventario.

1. `procesarAcciones(nombreArchivo)`: lee el archivo de texto de nombre `nombreArchivo` (un String) y lo procesa. Cada una de las líneas del archivo contiene una instrucción para agregar material al inventario o para construir una estructura. Por ejemplo, partiendo de un inventario vacío se procesan las siguientes instrucciones del archivo:

   ```
   tablas,3
   rocas,2
   tablas,2
   lingotes,1
   rocas,1
   muralla,madera
   lingotes,4
   escalera,metal
   ```

   Primero, se recopilan 3 tablas de madera, luego 2 rocas, luego 2 tablas de madera, luego 1 lingote de metal, luego 1 roca, luego se intenta

construir una muralla de madera (se puede, hay suficiente madera en el inventario), luego se recopila 4 lingotes de metal, y finalmente se intenta construir una escalera de metal (se puede). El inventario final después de todas estas instrucciones queda como:

```
[('piedra',2), ('piedra', 1), ('metal', 2)]
```

Ud. debe implementar procedimientos y funciones auxiliares, en caso que lo requiera. Para el testing, cree un programa de prueba partiendo con un inventario vacío y con al menos 20 instrucciones (de recopilación de material y construcción de estructuras), y luego verifique que el inventario es correcto después de cada instrucción. Para el testing del procedimiento `procesarAcciones`, invente al menos cuatro archivos de prueba con al menos 20 instrucciones cada uno, y verifique que, partiendo de un inventario vacío, termina con el inventario correcto luego de procesar todas las instrucciones del archivo. Incluya en todos los tests algunas instrucciones no válidas, para verificar que sus procedimientos las ignoran.

Parte 2

Implemente esta parte en el módulo `Tarea3Parte2.py`.

La importante empresa desarrolladora de videojuegos ha decidido empezar a utilizar Programación Orientada al Objeto para la programación de sus juegos, por lo que le solicita a Ud. reimplementar lo solicitado en la Parte 1 con las siguientes consideraciones:

- Debe crear la clase `Material`, que represente a los materiales.
- Debe crear la clase `Inventario`, que represente al inventario de un jugador. Debe tener como atributo la lista de Python con los materiales (que ahora son objetos de la clase `Material`), y los tres procedimientos definidos en la Parte 1 ahora deben ser métodos de esta clase.
- Debe crear la clase `Test`, que ejecute todos los tests de prueba de la clase `Inventario`, siguiendo la misma metodología de testing definida en la Parte 1 (puede reutilizar los tests que haya definido para la Parte 1).

Finalmente, escriba un programa principal que cree un objeto de la clase `Test` y utilícelo para invocar los tests definidos.

Consideraciones generales

Ud. debe implementar en forma obligatoria todos los procedimientos, funciones y métodos solicitados. Además, Ud. debe implementar todas las funciones o procedimientos auxiliares que requiera, en caso por ejemplo que para implementar algunas de las funciones o procedimientos solicitados necesite descomponer el problema en subproblemas más pequeños. Recuerde que cada función o procedimiento implementado debe resolver un único problema o realizar una única tarea, si está intentando resolver más de un problema a la vez piense en cómo se podría descomponer en subproblemas (esto será evaluado).

Asignación de puntajes

- Contrato y descripción: 0.5 puntos (repartido entre todas las funciones, procedimientos y métodos solicitados).
  - Contrato: 0.3 ptos. en total
  - Descripcion: 0.2 ptos. en total
- Parte 1: 3.5 ptos en total.
  - `agregarInventario`: 0.3 ptos.
  - `construirEstructura`: 1.5 ptos.
  - `procesarAcciones`: 0.7 ptos.
  - Testing: 1.0 ptos. (según lo definido en el enunciado)
- Parte 2: 2.0 ptos en total.
  - Clase `Material`: 0.3 ptos.
  - Clase `Inventario`: 1.2 ptos.
  - Testing: 0.5 ptos. (según lo definido en el enunciado)
- Descuentos globales:
  - Uso de buenas metodologías de programación: hasta -0.5 ptos.
  - Descomposición funcional: hasta -0.5 ptos.

Planificación para realizar la tarea

Al momento de la publicación de este enunciado, Ud. no dispone aún de todas las herramientas de programación para resolver la tarea completa, pero si las tendrá para la semana previa a la entrega.

Hay cuatro semanas de plazo para implementar todo lo solicitado en esta tarea. Se le recomienda la siguiente planificación para ir avanzando parcialmente en la tarea cada semana:

- Leer y entender bien el enunciado: Semana 1.
- Parte 1: Semanas 1 a 3.
- Parte 2: Semana 4.

Notas importantes

- Para programar su tarea, sólo puede utilizar las herramientas de Python que hayamos visto en el curso. En particular, no se pueden usar instrucciones de Python que no se hayan visto en las clase de cátedra, no estén en el apunte, o no hayan sido especificadas en este enunciado. Soluciones que no cumplan con este punto, serán evaluadas con la nota mínima.
- Se evaluará el uso de buenas metodologías de programación (receta de diseño, nombres de funciones y variables representativos, evitar duplicar código, etc.).
- Antes de ponerse a programar, piense en cuál sería una descomposición funcional adecuada para cada función solicitada.
- Esta prohibido utilizar ambientes de desarrollo con IA integrada, por ejemplo Colab. Utilice IDLE que se instala junto con Python para programar su tarea.
- Su tarea debe respetar *estrictamente* los nombres de funciones, procedimientos, parámetros, tipos de datos y reglas descritos en el enunciado.
- No olvide leer y seguir las reglas descritas en el documento de Política de Colaboración publicado en U-Cursos.

- Dudas sobre el enunciado pueden postearlas en el foro del curso. *ESTÁ PROHIBIDO COPIAR CÓDIGO FUENTE DE SU TAREA EN EL FORO*. Si Ud. copia código de su tarea en el foro, aunque sea para preguntar por un error, tendrá nota 1.0.
- La tarea debe entregarse a través de la página de U-Cursos, sección Tareas. Entregas durante el período de entrega con atraso no tienen descuento. *NO SE ACEPTARÁN TAREAS FUERA DE PLAZO NI ENTREGADAS POR OTROS MEDIOS.*